

What is a configmap?

- Kubernetes Objects for injecting containers with configuration data
- They hold configuration in key-value pairs
- ConfigMaps allow you to keep containers agnostic of specific configurations so they are more reusable.

When do ConfigMaps get used?

- Data from a ConfigMap is injected into the container at the time the POD is created.
- Any change made to an existing ConfigMap will ONLY effect NEWLY created PODS.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: special-config
  namespace: default
data:
  special.how: very
---
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: env-config
  namespace: default
data:
  log_level: INFO
```

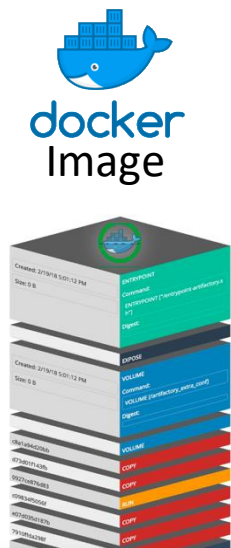
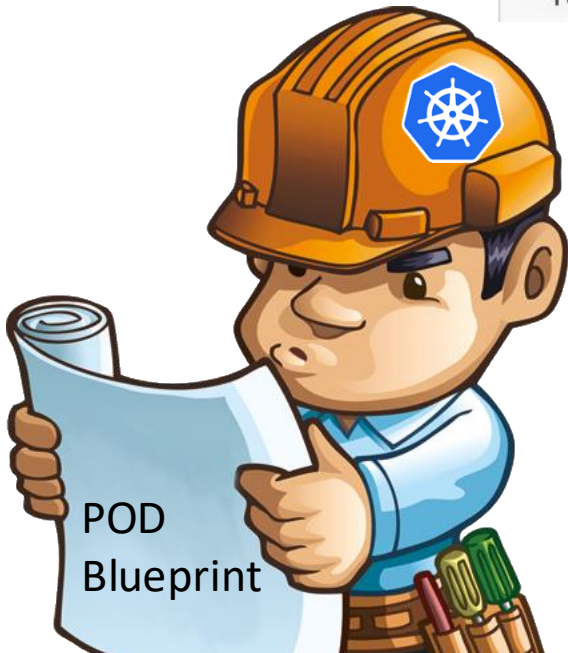
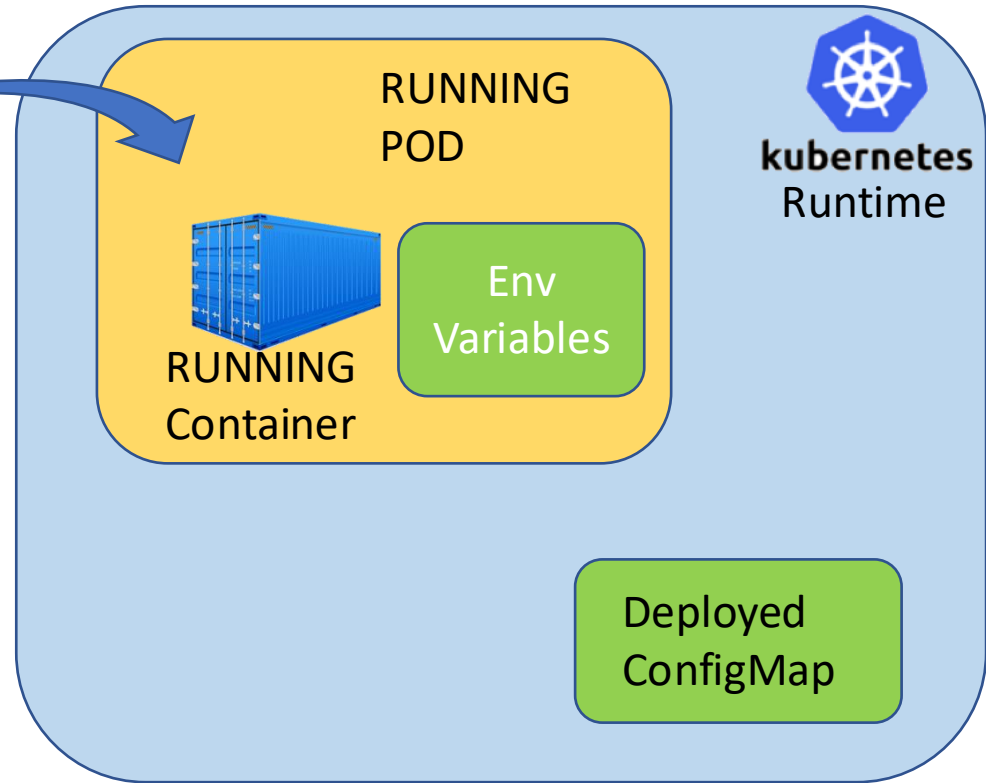
```
apiVersion: v1
kind: Pod
metadata:
  name: dapi-test-pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      command: [ "/bin/sh", "-c", "env" ]
      env:
        - name: SPECIAL_LEVEL_KEY
          valueFrom:
            configMapKeyRef:
              name: special-config
              key: special.how
        - name: LOG_LEVEL
          valueFrom:
            configMapKeyRef:
              name: env-config
              key: log_level
      restartPolicy: Never
```

kubernetes ConfigMap YAML

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: special-config
  namespace: default
data:
  special.how: very
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: env-config
  namespace: default
data:
  log_level: INFO
```

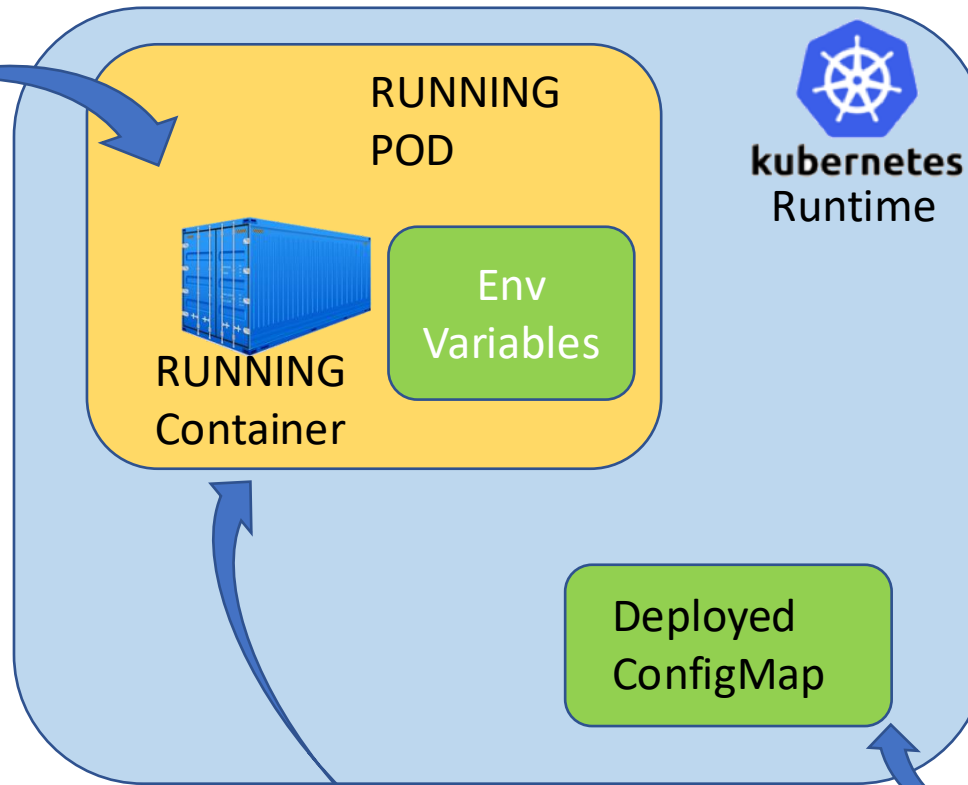
```
apiVersion: v1
kind: Pod
metadata:
  name: dapi-test-pod
spec:
  containers:
    - name: test-container
      image: k8s.gcr.io/busybox
      command: [ "/bin/sh", "-c", "env" ]
      env:
        - name: SPECIAL_LEVEL_KEY
          valueFrom:
            configMapKeyRef:
              name: special-config
              key: special.how
        - name: LOG_LEVEL
          valueFrom:
            configMapKeyRef:
              name: env-config
              key: log_level
      restartPolicy: Never
```

kubernetes POD YAML



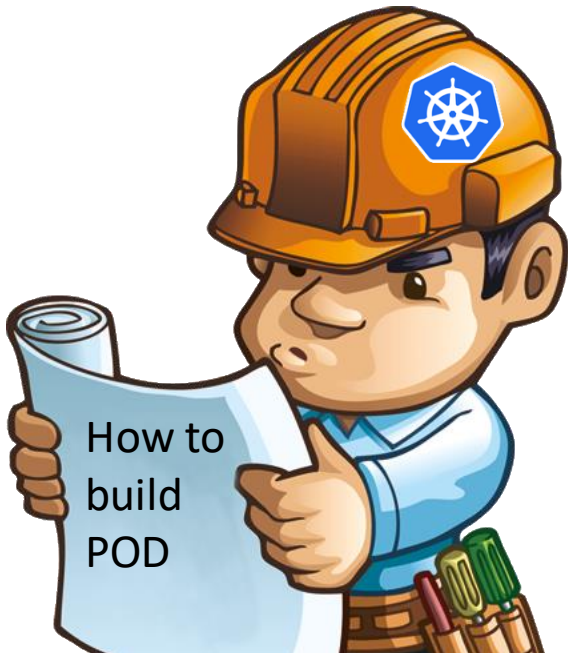
```
apiVersion: v1
kind: Pod
metadata:
  name: dapi-test-pod
spec:
  containers:
  - name: test-container
    image: k8s.gcr.io/busybox
    command: [ "/bin/sh", "-c", "env" ]
    env:
    - name: SPECIAL_LEVEL_KEY
      valueFrom:
        configMapKeyRef:
          name: special-config
          key: special.how
    - name: LOG_LEVEL
      valueFrom:
        configMapKeyRef:
          name: env-config
          key: log_level
  restartPolicy: Never
```

kubernetes
POD YAML



```
apiVersion: v1
kind: ConfigMap
metadata:
  name: special-config
  namespace: default
data:
  special.how: very
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: env-config
  namespace: default
data:
  log_level: INFO
```

kubernetes
ConfigMap YAML

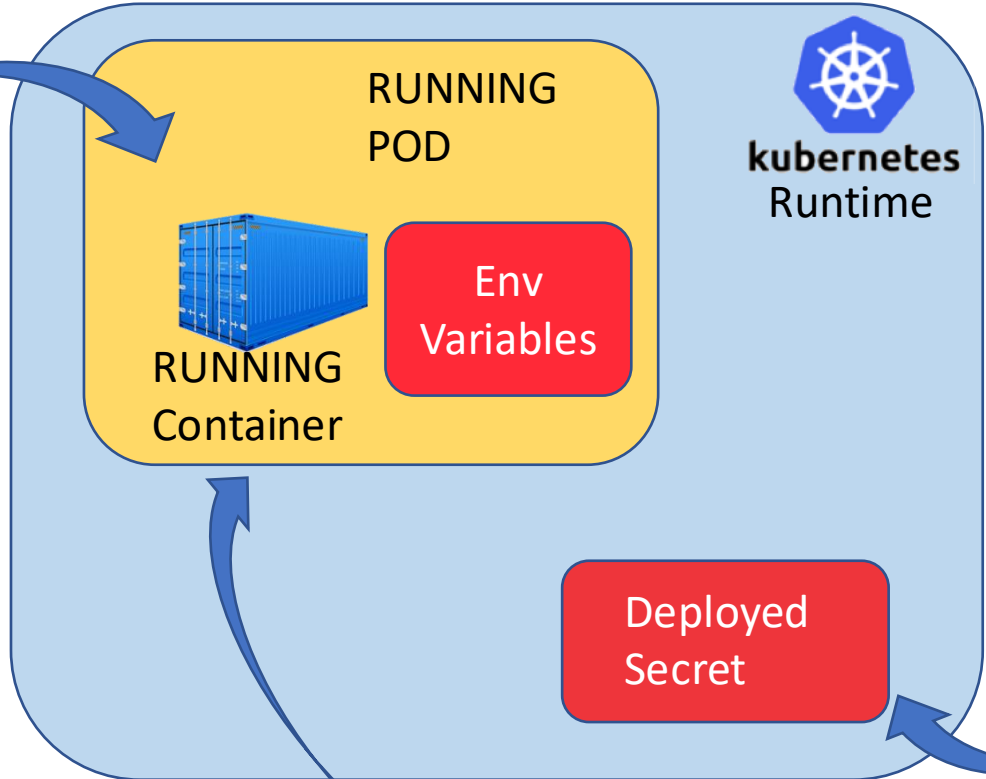


POD YAML



kubernetes

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-env-pod
spec:
  containers:
  - name: mycontainer
    image: redis
    env:
    - name: SECRET_USERNAME
      valueFrom:
        secretKeyRef:
          name: mysecret
          key: username
    - name: SECRET_PASSWORD
      valueFrom:
        secretKeyRef:
          name: mysecret
          key: password
    restartPolicy: Never
```

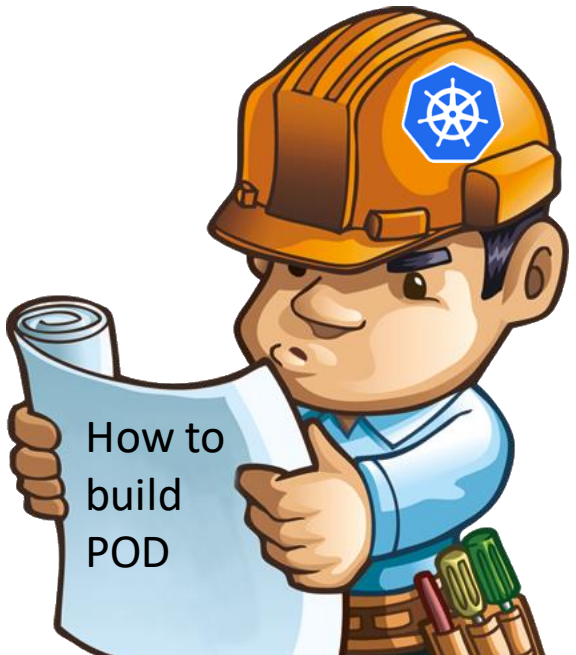


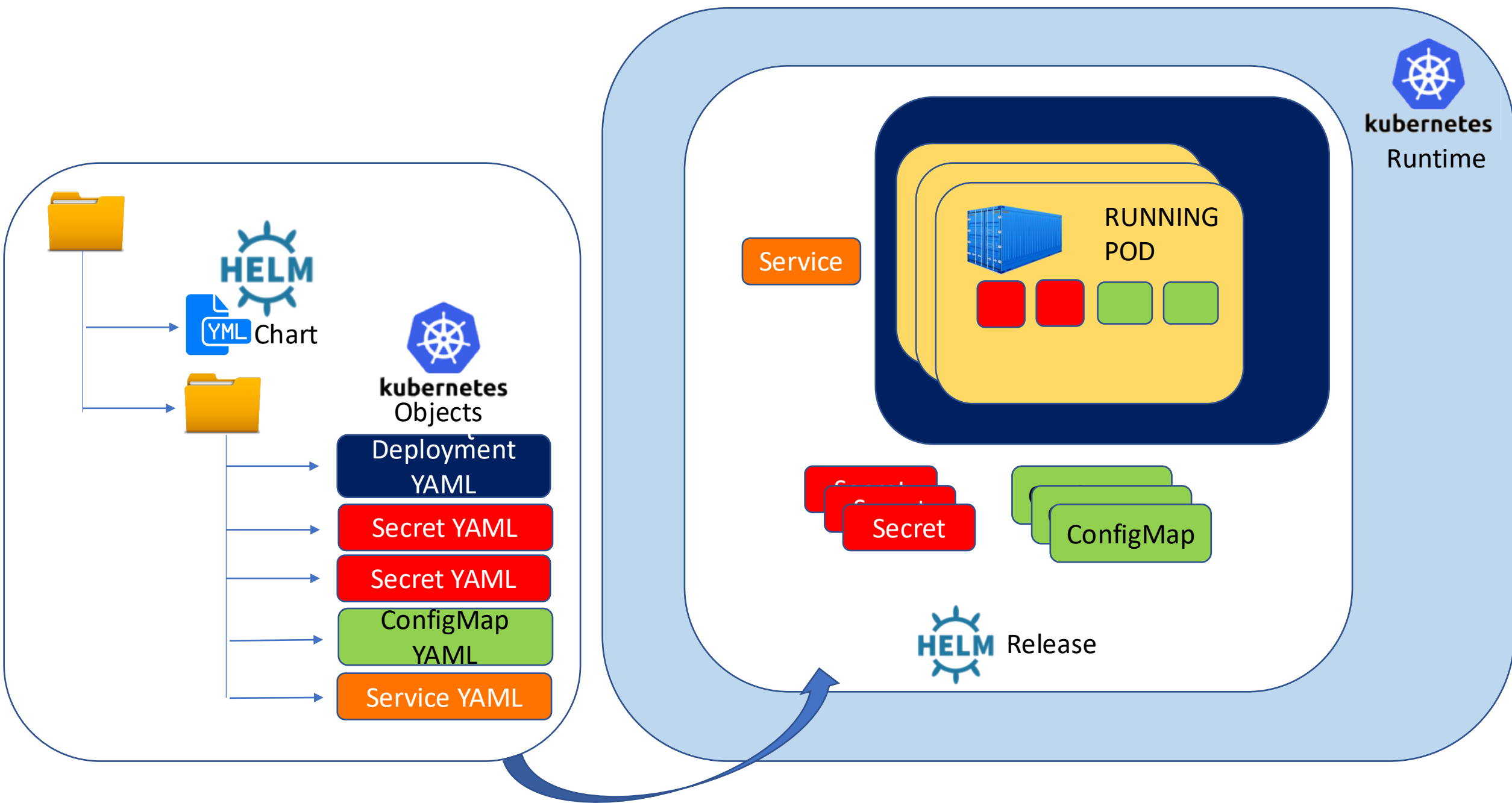
Secret YAML



kubernetes

```
apiVersion: v1
kind: Secret
metadata:
  name: mysecret
type: Opaque
data:
  username: YWRtaW4=
  password: MWYyZDF1MmU2N2Rm
```



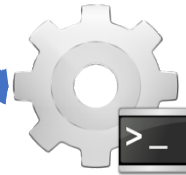




adminPassword.txt
appPassword.txt
keystore-mykey.crt
keystore-mykey.key
keystore-mykey.pass
keystorePassword.txt
mqsc.txt
odbc.ini
policy.xml
policyDescriptor.xml
serverconf.yaml
setdbparms.txt
truststoreCert-
mykey.crt
truststorePassword.txt



generateSecrets.sh



`./generateSecrets.sh mysecret`



kubernetes

Secret YAML

```
apiVersion: v1
kind: Secret
metadata:
  name: mysecret
type: Opaque
data:
  username: YWRtaW4=
  password: MWYyZDF1MmU2N2Rm
```

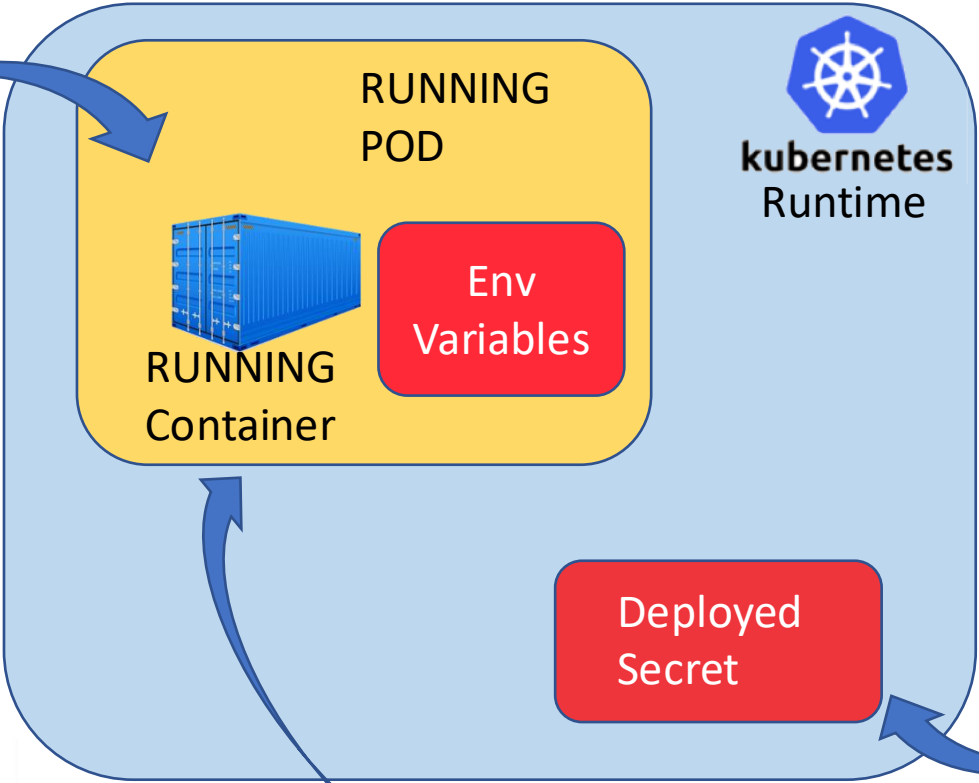
The script will look for any files detailed in the Configuration section in the same directory and if found will add them into keys in the generated secret. The script will then use the kubectl command to generate a secret containing the specified configuration for the integration server.

POD YAML



kubernetes

```
apiVersion: v1
kind: Pod
metadata:
  name: secret-env-pod
spec:
  containers:
  - name: mycontainer
    image: redis
    env:
    - name: SECRET_USERNAME
      valueFrom:
        secretKeyRef:
          name: mysecret
          key: username
    - name: SECRET_PASSWORD
      valueFrom:
        secretKeyRef:
          name: mysecret
          key: password
  restartPolicy: Never
```

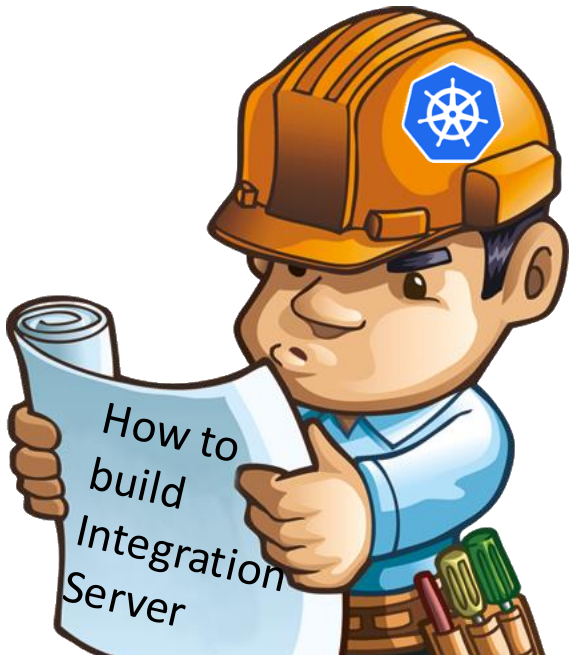




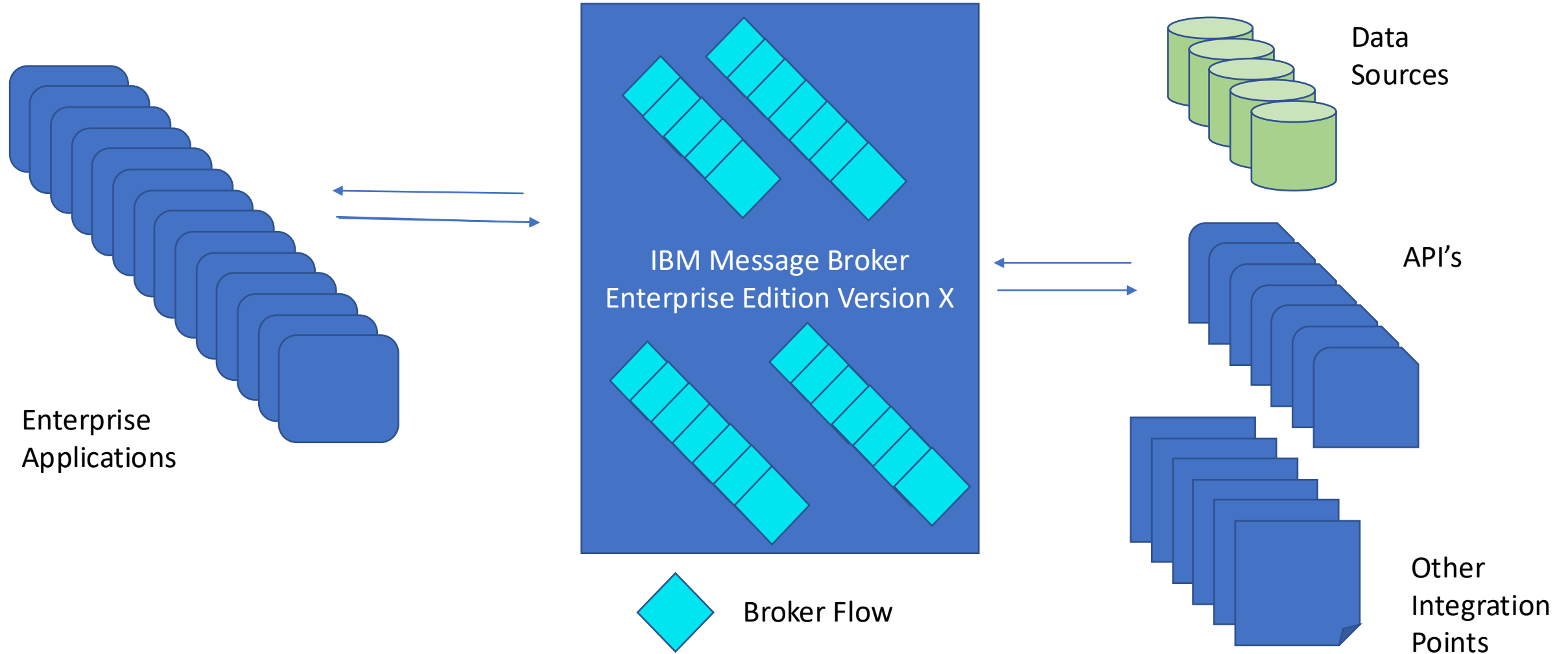
kubernetes

Secret YAML

```
apiVersion: v1
kind: Secret
metadata:
  name: mysecret
type: Opaque
data:
  username: YWRtaW4=
  password: MWYyZDF1MmU2N2Rm
```



All Broker Flows are hosted on a single instance

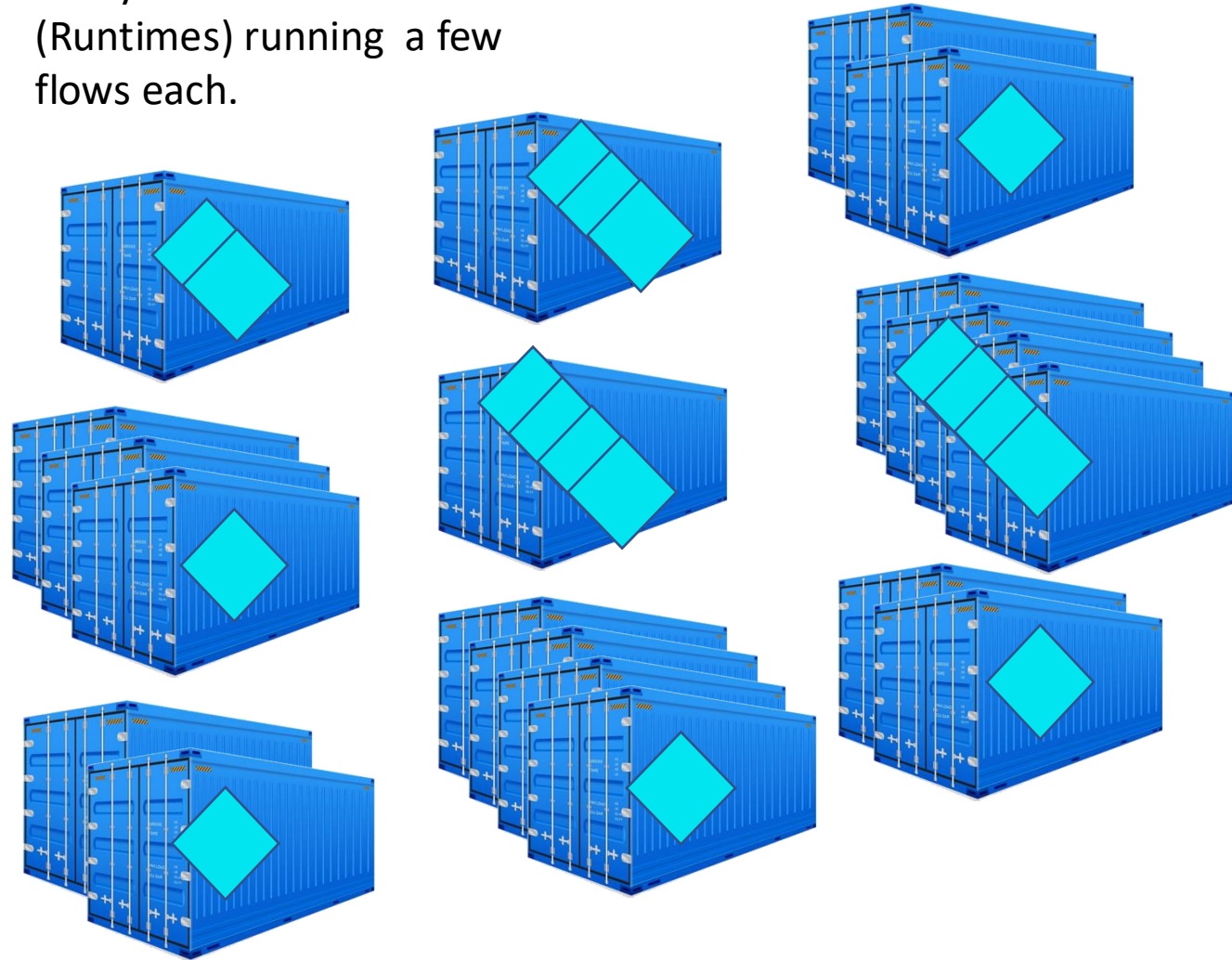


With Kubernetes – Things change

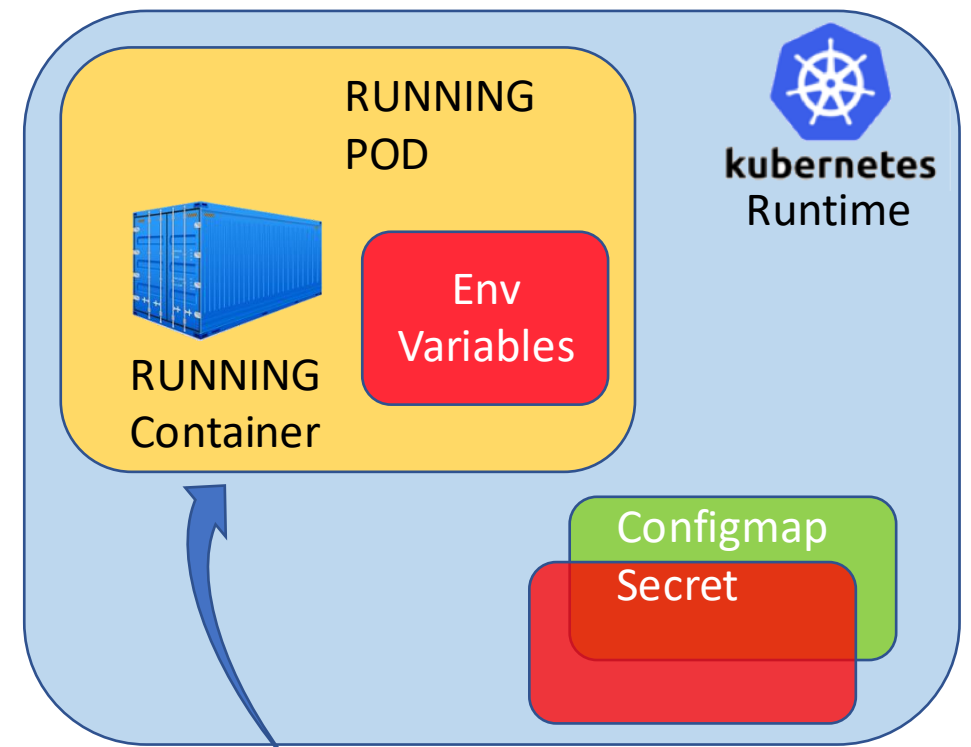
One Runtime – Many
Flows



Many Containers
(Runtimes) running a few
flows each.



- Many containers
 - Each with a unique set of Broker Flows
 - Some may be running different versions of IBM Integration Server
-
- DEVOPS – Very Important..



Set of Integration
Flows

IBM App
Managemnt
Runtime
Version X



Big investment in
the beginning

Time & Effort

Today's Challenges

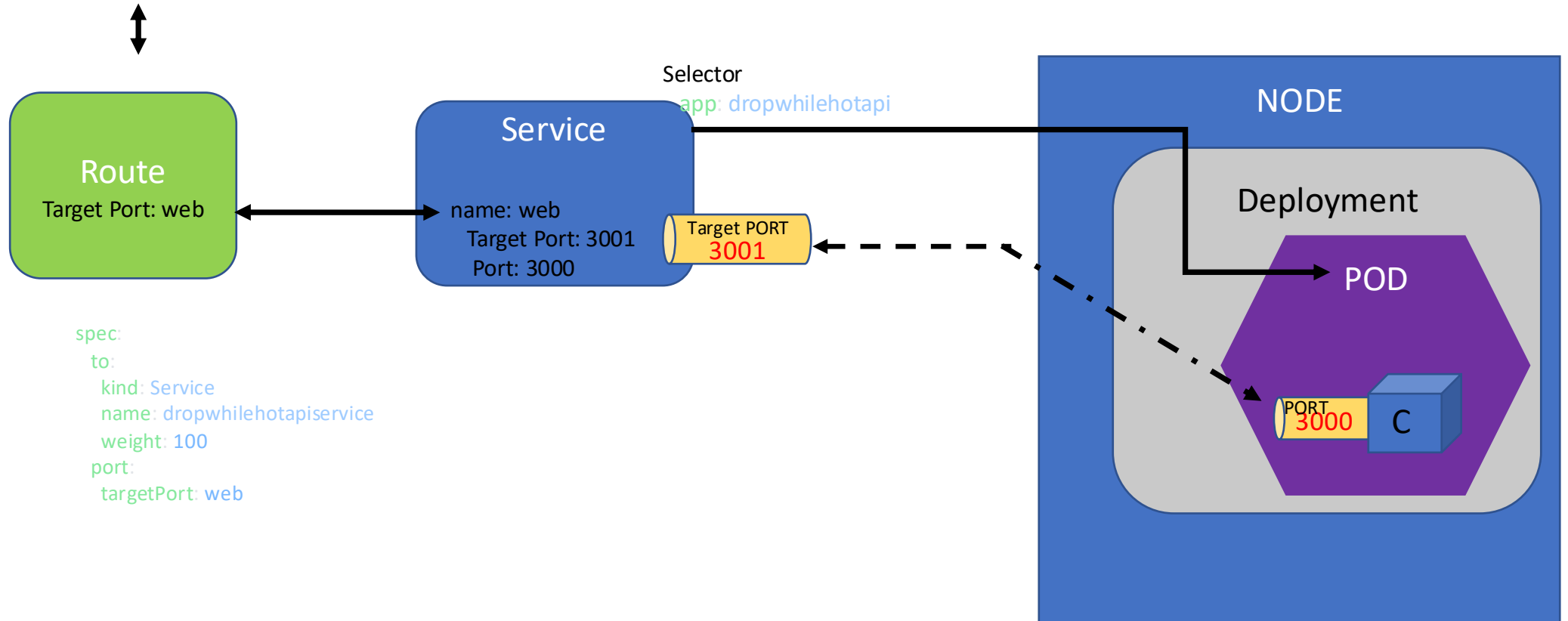
- Lack of skills
- Lack of dev/ops tools
- Culture "I have done this for years"
- Funding

Return on investment will take time

- Lower Risk Upgrades
- Ease of management
- Deployment of multiple versions
- Resiliency is high
- Portable deployments

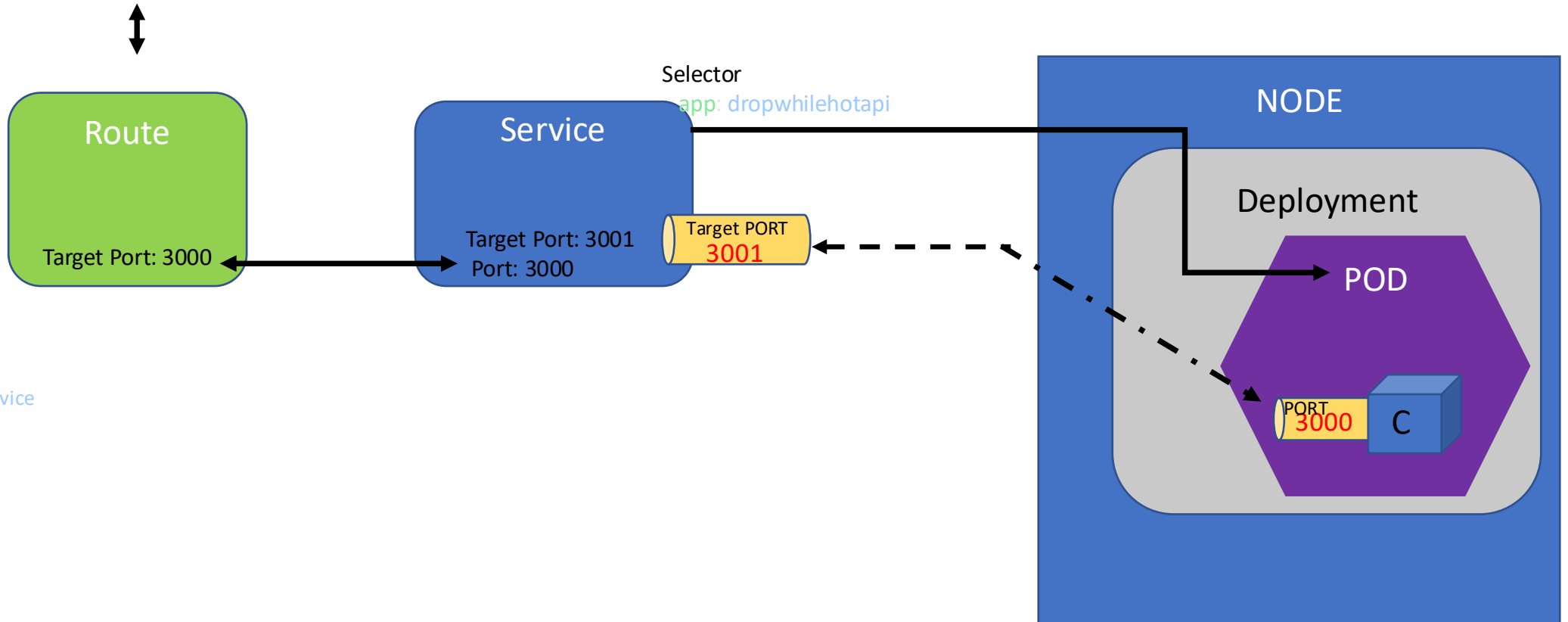
OpenShift Networking Service -> Route

spec:
host: dropwhilehot-route-default.apps.gamma03.kvm01.daffy-installer.com



OpenShift Networking Service -> Route

spec:
host: dropwhilehot-route-default.apps.gamma03.kvm01.daffy-installer.com



spec:
to:
kind: Service
name: dropwhilehotapiservice
weight: 100
port:
targetPort: 3000